

MSSV:23127195

Họ và tên : Trần Mạnh Hùng

Câu 1: Quyết định cơ chế xác thực cho Hệ thống Quản lý Sinh viên

1. Kết luận của em

Em đồng tình với **Nhóm A: Chọn Session-based authentication** làm phương án xác thực chính cho hệ thống quản lý sinh viên. Đồng thời, em phản đối quan điểm của Nhóm C. Việc chỉ dùng tài khoản/mật khẩu (dù đã được bảo vệ bằng cơ chế băm **hash + salt** như bcrypt hay Argon2) là chưa đủ an toàn trước các cuộc tấn công hiện đại. Việc áp dụng **MFA (Xác thực đa yếu tố)** là cần thiết, nhưng không nên bắt buộc một cách cứng nhắc cho toàn bộ người dùng trong mọi thao tác.

2. Lập luận bảo vệ kết luận

Hệ thống quản lý đại học chứa nhiều dữ liệu nhạy cảm (điểm số, thông tin cá nhân, học phí). Vì vậy, tính kiểm soát và khả năng xử lý sự cố bảo mật phải được đặt lên hàng đầu:

- **Tại sao chọn Session (Stateful) thay vì JWT (Stateless):** Session lưu trạng thái đăng nhập của người dùng trên server (**stateful**). Đặc điểm này mang lại một ưu điểm "sống còn" đối với hệ thống đại học: khả năng **revoke (thu hồi) quyền truy cập ngay lập tức**. Khi phát hiện một tài khoản có hành vi bất thường, hoặc khi sinh viên báo mất máy tính, quản trị viên có thể xóa session ID trên server, buộc tài khoản đó đăng xuất ngay lập tức ở mọi thiết bị. Ngược lại, JWT là **stateless** (lưu ở client), server chỉ kiểm tra chữ ký mà không lưu trạng thái, nên rất khó để revoke JWT giữa chừng trước khi token này tự hết hạn (expire).
- **Hai trade-off (sự đánh đổi) khi sử dụng Session:**
 - *Tiêu tốn bộ nhớ server:* Với một trường đại học có hàng chục ngàn sinh viên, server phải tốn một lượng tài nguyên RAM/Database đáng kể để lưu trữ và quản lý hàng chục ngàn Session ID đang active.
 - *Khó khăn trong việc mở rộng (Scale):* Khi hệ thống cần **scale** ngang (thêm nhiều server để gánh tải), các server này phải chia sẻ chung một kho lưu trữ session (thường dùng Redis). Việc cấu hình và duy trì kho lưu trữ tập trung này phức tạp hơn so với JWT (nơi mỗi server tự xác thực độc lập).
- **Đánh giá việc áp dụng MFA:** Việc áp dụng MFA nên dựa trên mức độ nhạy cảm của hành động thay vì ép buộc tất cả:
 - *Không bắt buộc MFA* cho các tác vụ đọc thông thường như xem thời khóa biểu, xem thông báo để đảm bảo trải nghiệm người dùng (UX) tốt.
 - *Bắt buộc MFA* đối với các tài khoản có đặc quyền (Giảng viên nhập/sửa điểm, Quản trị viên hệ thống) hoặc khi người dùng thực hiện các giao dịch quan trọng (Sinh viên thanh toán học phí, đổi mật khẩu).

3. Một phản biện ngược lại

Một kỹ sư ủng hộ Nhóm B (JWT) có thể phản biện như sau: *"Hệ thống đại học hiện nay thường được xây dựng theo kiến trúc phân tán (nhiều microservices độc lập như: Dịch vụ E-learning, Dịch vụ Thư viện, Dịch vụ Thanh toán). Việc dùng Session bắt buộc tất cả các dịch vụ này phải liên tục gọi về một cơ sở dữ liệu Session tập trung để xác thực request, tạo ra một 'điểm nghẽn' (bottleneck). Thay vào đó, dùng JWT với tính chất stateless sẽ giúp hệ thống dễ dàng **scale** ra nhiều node, mỗi service chỉ cần dùng Secret Key để tự verify token mà không cần tốn chi phí gọi network về database."*

4. Một ví dụ hoặc tình huống cụ thể

- **Tình huống phương án em không chọn (JWT) lại phát huy hiệu quả tốt hơn:** Trong ngày **Đăng ký học phần**, lưu lượng sinh viên truy cập đồng thời tăng đột biến gấp hàng trăm lần so với ngày thường. Nếu dùng Session-based, kho lưu trữ tập trung (như Redis) có thể bị quá tải nghiêm trọng do phải xử lý hàng triệu lệnh đọc/ghi session mỗi phút, dẫn đến sập toàn bộ hệ thống đăng ký. Nếu hệ thống sử dụng **JWT**, request của sinh viên sẽ được phân tải (load balancing) ra hàng chục server khác nhau. Các server này tự giải mã và kiểm tra tính hợp lệ của JWT ngay tại bộ nhớ cục bộ mà không cần truy vấn Database, giúp hệ thống chịu tải mượt mà hơn rất nhiều.

Câu 2: Lựa chọn mô hình phân quyền cho Hệ thống Quản lý Đề thi

1. Kết luận của em

Chỉ sử dụng **RBAC** là **hoàn toàn chưa đủ** để đáp ứng các điều kiện khắt khe của hệ thống quản lý đề thi. Hệ thống này bắt buộc phải áp dụng **ABAC** đối với các điều kiện về không gian, thời gian và đặc tính tài liệu. Thực tế, kiến trúc tối ưu nhất không phải là chọn 1 trong 2, mà là **mô hình kết hợp (Hybrid)**: Dùng RBAC làm nền tảng định tuyến vai trò cơ bản, và dùng ABAC làm màng lọc cuối cùng để kiểm soát ngữ cảnh truy cập.

2. Lập luận bảo vệ kết luận

- **Tại sao RBAC là chưa đủ:** RBAC chỉ phân quyền theo "định danh tĩnh" (Role - ai được làm gì). Nó giải quyết rất tốt bài toán: *"Giảng viên được ra đề, Tổ trưởng được duyệt đề"*. Tuy nhiên, nó hoàn toàn "mù" trước các **yếu tố ngữ cảnh động** như thời gian thực tế, vị trí mạng, hay mức độ mật của từng file.
- **Phản nào bắt buộc dùng ABAC:** Theo tài liệu, ABAC (Attribute-Based Access Control) cho phép quyết định dựa trên các **thuộc tính (attributes)**. Trong hệ thống này, các điều kiện sau phải dùng ABAC (được kiểm tra qua policy engine):
 - *Duyệt đề trong khung giờ quy định:* Kiểm tra thuộc tính Môi trường (Environment attribute - env.hour).

- *Xem từ mạng nội bộ*: Kiểm tra thuộc tính Môi trường (IP tin cậy - `env.ipTrusted`).
- *Tài liệu có mức độ nhạy cảm khác nhau*: Kiểm tra thuộc tính Tài nguyên (Resource attribute - `resource.sensitivity`).
- **Đề xuất chính sách phân quyền sơ bộ (Policy)**:
 - *Tầng 1 (RBAC)*: Gán role cho user. VD: `user.role === 'to_truong'`.
 - *Tầng 2 (ABAC rules)*:
 - *Policy duyệt đề*: NẾU `user.role == 'to_truong'` VÀ (`env.hour >= 8` VÀ `env.hour <= 17`) THÌ cho phép hành động approve.
 - *Policy xem tài liệu mật*: NẾU `action == 'read'` VÀ `resource.sensitivity == 'high'` THÌ bắt buộc `env.ipTrusted == true`.
- **Vì sao hệ thống thực tế phải phối hợp cả hai**: Nếu ép RBAC phải gánh các điều kiện ngữ cảnh, ta sẽ gặp rủi ro **"Role Explosion"** (bùng nổ vai trò), phải tạo ra hàng tá role rất nực cười như `ToTruong_TrongGio_MangNoiBo`, khiến hệ thống không thể quản lý nổi. Ngược lại, nếu bỏ RBAC để dùng 100% ABAC ngay từ đầu, việc thiết lập các quy tắc if-then cho mọi API sẽ vô cùng phức tạp và hao tốn tài nguyên xử lý (evaluate). Phối hợp cả hai giúp dễ quản trị tổng thể (RBAC) nhưng vẫn bảo mật chặt chẽ ở chi tiết (ABAC).

3. Một phản biện ngược lại

Một số lập trình viên có thể phản biện: *"Việc code thêm một policy engine cho ABAC (với các interface Context, check if-then liên tục) sẽ làm tăng độ trễ (latency) của API và phức tạp hóa source code. Thay vì dùng ABAC trong code, ta có thể dùng cấu hình mạng hạ tầng (Ví dụ: Firewall chặn mọi IP bên ngoài truy cập vào router duyệt đề, và dùng CronJob để tắt API Server ngoài giờ hành chính). Dùng hạ tầng giải quyết sẽ giúp giữ nguyên RBAC thuần túy cho phần mềm."*

(Tuy nhiên, cách làm hạ tầng này lại cực kỳ cứng nhắc. Khảo thí hoặc Quản trị viên khi cần xử lý sự cố ngoài giờ hoặc làm việc từ xa sẽ bị block hoàn toàn, không linh hoạt như một hệ thống ABAC được thiết kế tốt).

4. Một ví dụ hoặc tình huống cụ thể

- **Tình huống áp dụng ABAC**: Cô C là **Tổ trưởng bộ môn** (Thỏa mãn Role của RBAC). Vào lúc 22:00 tối Chủ nhật, cô C ngồi tại một **quán cà phê wifi công cộng** và định dùng laptop cá nhân để tải về một đề thi cuối kỳ có đánh dấu **"Tuyệt mật" (high sensitivity)** để xem trước.
 - Nếu hệ thống chỉ có **RBAC**: Hành động tải file thành công vì cô C có role là Tổ trưởng. (Nguy cơ lộ đề thi ra mạng công cộng rất lớn).
 - Khi có **ABAC**: Middleware của hệ thống lập tức bắt lại request. Nó phát hiện `env.hour (22:00)` vi phạm giờ làm việc, và `env.ipTrusted (IP quán cà phê)`

trả về false đối với tài liệu sensitivity: 'high'. Hệ thống sẽ quăng lỗi 403 Forbidden ngay lập tức, bảo vệ an toàn tuyệt đối cho đề thi dù tài khoản của người dùng có quyền cực kỳ cao.

Câu 3: Đánh giá các ngộ nhận về bảo mật (OAuth2, CORS, API Key)

1. Kết luận của em

Cả ba phát biểu của nhóm sinh viên đều chứa những ngộ nhận sai lầm và cực kỳ nguy hiểm. Nếu xây dựng hệ thống dựa trên những tư duy này, ứng dụng sẽ ngay lập tức đối mặt với rò rỉ dữ liệu (data breach) và thiệt hại tài chính. Cụ thể:

- OAuth2 là **ủy quyền (Authorization)**, không phải xác thực (Authentication).
- CORS là **bảo mật trình duyệt (Browser Security)**, không phải là bức tường bảo vệ API.
- Nhúng API Key vào client là hành động rất nguy hiểm về mặt bảo mật.

2. Lập luận bảo vệ kết luận

Trước tiên, cần phân biệt rạch ròi 3 khái niệm mà nhóm sinh viên đang nhầm lẫn:

- **Authentication (Xác thực):** Trả lời câu hỏi "*Bạn là ai?*" (VD: Đăng nhập bằng tài khoản/mật khẩu).
- **Authorization (Phân quyền/Ủy quyền):** Trả lời câu hỏi "*Bạn được phép làm gì?*" (VD: App A được quyền đọc danh bạ của bạn trên Google).
- **Browser Security:** Cơ chế bảo vệ người dùng cuối do trình duyệt áp đặt để tránh các trang web độc hại tấn công chéo.

Đánh giá từng phát biểu:

- **Phát biểu 1: "OAuth2 là cơ chế đăng nhập."**
 - **Sai ở đâu:** OAuth2 sinh ra không phải để định danh người dùng. Nó là chuẩn **ủy quyền (Authorization)**, cấp cho ứng dụng thứ 3 một cái thẻ (access token) có giới hạn phạm vi (scope) để thao tác dữ liệu thay người dùng.
 - **Hậu quả kỹ thuật:** Nếu dùng OAuth2 thuần túy để "đăng nhập", hệ thống rất dễ dính lỗi *Confused Deputy Problem*. Kẻ tấn công có thể dùng một access token hợp lệ (nhưng được cấp cho một ứng dụng khác) để đánh lừa ứng dụng của bạn và đăng nhập trái phép vào tài khoản nạn nhân.
 - **Thiết kế an toàn:** Để dùng OAuth2 cho mục đích đăng nhập, phải sử dụng **OpenID Connect (OIDC)**. Đây là một lớp xác thực (Authentication) nằm phía trên OAuth2, cung cấp thêm ID Token để hệ thống biết chính xác người dùng là ai.
- **Phát biểu 2: "CORS cấu hình đúng là đủ để bảo vệ API."**

- **Đúng/Sai ở đâu:** Đúng một nửa là CORS giúp chặn các domain lạ gọi API trên trình duyệt. Nhưng sai hoàn toàn ở chữ "đủ". Trình duyệt tuân thủ chính sách *Same-origin policy*, còn máy chủ thì không.
- **Hậu quả kỹ thuật:** Hacker **không bao giờ** dùng trình duyệt (Chrome/Edge) để đi hack API. Họ dùng Postman, cURL, hoặc viết script (Python/Node.js) để gọi thẳng vào API của bạn. Các công cụ này bỏ qua hoàn toàn cơ chế CORS. Nếu API không có phân quyền, hacker sẽ lấy được toàn bộ database.
- **Thiết kế an toàn:** CORS chỉ là lớp áo giáp ngoài cùng trên web. Mọi API đều phải được bảo vệ tại **Backend** bằng **Authentication (Session/JWT)** và **Authorization (RBAC/ABAC)**.
- **Phát biểu 3: "Có thể nhúng API key vào frontend/mobile app vì người dùng bình thường không thấy được."**
 - **Sai ở đâu:** Rất ngây thơ. Người dùng bình thường không thấy, nhưng lập trình viên hoặc hacker thì dư sức. Mã nguồn Frontend (React, Vue) khi tải về trình duyệt ai cũng có thể xem được. Mobile App (APK, IPA) cũng dễ dàng bị dịch ngược (reverse engineering).
 - **Hậu quả kỹ thuật:** Hacker sẽ lấy được API Key. Nếu đây là key của dịch vụ trả phí (như AI, SMS, Cloud Storage), hacker sẽ spam request, làm hệ thống cạn kiệt tài chính (billing attack) và lấy cắp toàn bộ dữ liệu đang lưu tại dịch vụ đó.
 - **Thiết kế an toàn:** API Key phải được lưu ở **Backend** (thông qua biến môi trường - Environment Variables). Frontend muốn gọi dịch vụ thì phải gọi lên Backend của mình (có kèm token xác thực người dùng), sau đó Backend mới dùng API Key bí mật kia để gọi sang dịch vụ bên thứ ba.

3. Một phản biện ngược lại

Một bạn sinh viên trong nhóm có thể phản biện lại lý thuyết của em như sau: *"Dự án của bọn em chỉ là làm cho xong đồ án môn học, không có budget để thuê server chạy Backend. Bọn em dùng Firebase (BaaS) và gọi thẳng các API AI từ code React (Frontend). Để tránh lộ key, bọn em đã dùng các công cụ mã hóa (obfuscate code) dập nát source code rồi mới build ra. Việc tạo thêm Backend trung gian chỉ để giấu cái API Key là quá tốn công sức, làm over-engineering (phức tạp hóa) một bài tập nhỏ."*

(Tuy nhiên, cần hiểu rằng mã hóa code (obfuscate) chỉ làm khó người đọc chứ không giấu được dữ liệu. Khi request được gửi đi từ trình duyệt, cái API Key đó bằng cách nào đó vẫn phải nằm "chính ình" trong tab Network. Thói quen "làm cho nhanh" này nếu mang ra doanh nghiệp sẽ trả giá rất đắt).

4. Một ví dụ hoặc tình huống cụ thể

Tình huống "Hóa đơn nghìn đô" vì ngộ nhận CORS và API Key: Nhóm D làm một website sửa lỗi ngữ pháp tiếng Anh, dùng API trả phí của OpenAI. Vì tin rằng *"CORS là đủ bảo vệ"*, nhóm chỉ cấu hình CORS cho phép domain website-nhomD.com được gọi API. Vì tin rằng *"Front-end không ai thấy code"*, nhóm nhúng thẳng API Key của OpenAI vào mã nguồn React. Một ngày nọ, một người dùng táy máy bấm phím F12 (Developer Tools), sang tab

Network, xem được cái request và **copy cái API Key** đó. Người này mở ứng dụng Postman lên (nơi mà CORS vô tác dụng) và thiết lập một kịch bản gọi API tự động để render hàng ngàn bài báo. Sáng hôm sau, nhóm D nhận được hóa đơn trị giá \$5,000 từ OpenAI do số lượng request tăng đột biến, và dự án bị khóa ngay lập tức.

Dưới đây là bài phân tích chi tiết cho **Câu 4**, mô tả chiến lược Rate Limiting (Giới hạn lưu lượng) cho hệ thống thực tế theo đúng cấu trúc 4 phần.

Câu 4: Thiết kế chiến lược Rate Limiting cho API `/login` và `/generate-report-ai`

1. Kết luận của em

Tuyệt đối không sử dụng chung một chính sách Rate Limit cho cả hai API này.

- Đối với `/login`, em chọn thuật toán **Sliding Window** (Cửa sổ trượt) với khóa đếm kết hợp **IP + User ID**.
- Đối với `/generate-report-ai`, em chọn thuật toán **Token Bucket** (Xô Token) với khóa đếm theo **User ID** (hoặc API Token). Trong môi trường nhiều server, bắt buộc phải dùng **Distributed Rate Limiting** (đồng bộ qua Redis) chứ không được đếm cục bộ (local).

2. Lập luận bảo vệ kết luận

- **Vì sao không dùng chung một chính sách?**
 - Đặc thù hai API hoàn toàn trái ngược. `/login` tiêu tốn rất ít tài nguyên nhưng lại là mục tiêu của tấn công bảo mật (Brute-force mật khẩu). Trong khi đó, `/generate-report-ai` gọi các mô hình LLM, tiêu tốn chi phí tài chính (cost) khổng lồ và mất nhiều giây để xử lý.
 - Nếu dùng chung limit quá lỏng (VD: 100 req/phút): Hacker sẽ dễ dàng dò mật khẩu `/login`, hoặc làm công ty phá sản vì gọi `/generate-report-ai` 100 lần.
 - Nếu dùng chung limit quá chặt (VD: 3 req/giờ): API AI an toàn, nhưng người dùng gõ sai mật khẩu `/login` 3 lần sẽ bị khóa tài khoản cả ngày, trải nghiệm cực kỳ tệ.
- **Chọn thuật toán phù hợp:**
 - **`/login` → Sliding Window:** Đăng nhập cần sự chính xác tuyệt đối theo thời gian thực để chống dò mật khẩu. Nếu dùng Fixed Window (Cửa sổ cố định), hacker có thể dồn request vào giây thứ 59 và giây thứ 01 của phút tiếp theo để

vượt rào (hiện tượng burst tại ranh giới). Sliding Window tính toán cửa sổ trượt liên tục, chặn đứng mọi nỗ lực brute-force một cách chính xác nhất.

- **/generate-report-ai → Token Bucket:** API AI có thể cần tạo vài báo cáo cùng lúc rồi thôi. Token Bucket cho phép "burst ngắn hợp lý" (VD: bucket chứa tối đa 3 token, mỗi giờ hồi 1 token). Người dùng có thể tạo ngay 3 báo cáo liên tục khi cần gấp, nhưng sau đó phải đợi rất lâu mới được tạo tiếp. Nó cân bằng giữa UX và việc kiểm soát chi phí. (*Leaky bucket cũng là một lựa chọn tốt nếu muốn bảo vệ hoàn toàn server AI khỏi bị quá tải cục bộ*).
- **Đề xuất khóa đếm (Rate Limit Key):**
 - **/login → Kết hợp IP + User ID:** Không thể chỉ chặn theo User ID (hacker dùng botnet đổi IP liên tục để đánh 1 tài khoản), cũng không thể chỉ chặn theo IP (ký túc xá có hàng trăm sinh viên dùng chung 1 IP public, 1 người gõ sai sẽ khóa cả trường). Phải kết hợp cả hai hoặc tạo 2 layer (VD: Max 5 lần sai/User ID + Max 50 lần sai/IP).
 - **/generate-report-ai → User ID (hoặc API Token):** Tài nguyên AI tính tiền theo người dùng/khách hàng. Cho dù họ có đổi IP mạng hay sang máy khác, dung lượng (quota) của họ vẫn phải bị trừ thống nhất.
- **Xử lý Rate Limiting phân tán (Distributed):**
 - Khi chạy nhiều server, không thể lưu biến đếm trên RAM của từng Node. Phải sử dụng một kho lưu trữ tập trung, tốc độ cao như **Redis**. Mỗi khi có request, server sẽ gọi vào Redis để tăng biến đếm (dùng lệnh INCR hoặc các Lua script để đảm bảo tính nguyên tử - atomic). Dù người dùng hit vào server nào, Redis cũng sẽ trả về con số chính xác toàn cục.

3. Một phản biện ngược lại (Phản biện ý kiến: “mỗi server đếm local là đủ”)

Một lập trình viên thiếu kinh nghiệm có thể đề xuất: *"Việc mỗi request đều gọi qua Redis sẽ làm tăng độ trễ mạng (latency) và tạo ra thất cổ chai tại database. Chỉ bằng ta cấu hình mỗi server tự đếm local trong RAM của nó cho nhanh, không cần đồng bộ."*

Phản biện: Quyết định này phá vỡ hoàn toàn nghiệp vụ giới hạn lưu lượng trong kiến trúc phân tán. Giả sử hệ thống có 10 server (Node) đằng sau một Load Balancer (Bộ cân bằng tải), và giới hạn sinh báo cáo AI là 5 lần/ngày. Nếu đếm local, người dùng gửi request thứ 6, Load Balancer điều phối request đó sang Server số 2 (nơi biến đếm vẫn đang là 0). Kết quả: Người dùng có thể lách luật để gọi tối đa **50 lần/ngày** (5 lần x 10 server), làm chi phí vận hành AI đội lên gấp 10 lần. Trade-off (đánh đổi) thêm vài mili-giây độ trễ của Redis là hoàn toàn xứng đáng để cứu công ty khỏi một thảm họa tài chính.

4. Một tình huống cụ thể (Ví dụ về hậu quả của việc chọn sai khóa đếm)

Tình huống "Khóa nhầm cả công ty đối tác": Một nền tảng SaaS cung cấp công cụ AI cho các doanh nghiệp. Lập trình viên thiết lập Rate Limit cho `/generate-report-ai` là 10 báo cáo/giờ và chọn khóa đếm duy nhất là **theo IP** (vì nghĩ nó dễ cài đặt). Công ty đối tác X mua gói trả phí cho toàn bộ 50 nhân viên. Do hệ thống mạng của công ty X đều chui qua chung một

router (1 IP Public duy nhất), khi 10 nhân viên đầu tiên nhấn "Tạo báo cáo", API lập tức đạt giới hạn 10 req/IP. Kết quả: 40 nhân viên còn lại bị hệ thống báo lỗi 429 Too Many Requests trong suốt 1 tiếng đồng hồ, dù họ đã trả tiền bản quyền đầy đủ. Nếu lập trình viên thiết kế khóa đếm bằng **User ID / API Token** như kết luận ở trên, tình huống tồi tệ này đã không xảy ra.

Dưới đây là bài phân tích chi tiết cho **Câu 5**, giải quyết trọn vẹn bài toán học búa về thanh toán và đồng bộ dữ liệu theo đúng cấu trúc 4 phần.

Câu 5: Chiến lược xử lý trùng lặp và xung đột trong hệ thống thanh toán học phí

1. Kết luận của em

Để giải quyết triệt để hiện tượng mạng chập chờn và xung đột dữ liệu trong hệ thống thanh toán học phí, việc chỉ dùng Database Transaction là **không đủ**. Hệ thống này bắt buộc phải sử dụng **mô hình kết hợp**:

1. Sử dụng **Idempotency Key** ở tầng API để chặn đứng các request trùng lặp do người dùng bấm nhiều lần.
2. Sử dụng **Pessimistic Locking (Khóa bi quan)** ở tầng Database để bảo vệ tính nhất quán tuyệt đối của số dư tài khoản khi có nhiều luồng xử lý cùng lúc.

2. Lập luận bảo vệ kết luận

- **Vì sao Transaction thôi chưa chắc đã đủ?**

Transaction (Giao dịch DB) có tính chất ACID, đảm bảo một cụm thao tác (ví dụ: trừ tiền tài khoản + ghi log) hoặc là thành công tất cả, hoặc là thất bại toàn bộ. Tuy nhiên, nếu sinh viên bấm nút thanh toán 2 lần, hệ thống sẽ tạo ra **2 transaction độc lập và hoàn toàn hợp lệ**. Database không có cách nào biết được transaction thứ 2 là do người dùng "lỡ tay" bấm đúp. Kết quả là sinh viên vẫn bị trừ tiền 2 lần.

- **Khi nào cần Idempotency Key?**

Theo tài liệu, Idempotency được dùng cho mọi API **có khả năng retry** (gửi lại). Nó xử lý vấn đề ở tầng mạng/API. Khi client tạo ra một mã Idempotency-Key duy nhất cho một phiên thanh toán, server sẽ lưu mã này lại. Dù người dùng có bấm 10 lần do lag, server chỉ thực thi logic trừ tiền đúng 1 lần cho cái key đó, 9 lần sau nó chỉ trả về kết quả đã lưu.

- **Khi nào dùng Pessimistic vs Optimistic Locking?**

Hai cơ chế này giải quyết xung đột ở tầng Database (khi 2 process thực sự khác nhau muốn sửa cùng 1 dòng dữ liệu).

- **Pessimistic Locking (Khóa bi quan):** Phù hợp cho hệ thống tài chính, ngân hàng, nơi xung đột cao và yêu cầu nhất quán tuyệt đối (không cho phép sai số dư). Cơ chế này "khóa cửa" ngay khi đọc, ép process khác phải xếp hàng chờ.

- **Optimistic Locking (Khóa lạc quan):** Phù hợp cho web app thông thường (mạng xã hội, đếm view, sửa bài viết) nơi dữ liệu đọc nhiều, ghi ít, và xung đột hiếm xảy ra. Cơ chế này kiểm tra version lúc commit, nếu sai version thì báo lỗi bắt ứng dụng tự thử lại (retry).
- **Khi nào phải kết hợp Idempotency và Locking?**
Chính là **hệ thống thanh toán** này.
 - *Idempotency* bảo vệ user A khỏi việc tự trừ tiền chính mình 2 lần (do bấm đúp).
 - *Pessimistic Locking* bảo vệ user A khi có 2 luồng độc lập cùng tác động vào số dư của A (ví dụ: Sinh viên đang bấm thanh toán bằng thẻ, cùng lúc đó hệ thống tự động trừ tiền gia hạn bảo hiểm y tế).
- **Hệ quả nghiệp vụ nếu chọn sai cơ chế:**
 - *Thiếu Idempotency:* Trừ tiền khách hàng nhiều lần $\rightarrow$ Khách hàng hoang mang, kiện cáo, đội CSKH quá tải vì phải đi hoàn tiền (refund) thủ công, mất uy tín nghiêm trọng.
 - *Chọn Optimistic thay vì Pessimistic cho thanh toán:* Khi có xung đột (thanh toán cùng lúc), một giao dịch sẽ bị fail (văng lỗi rollback). Hệ thống phải tự động retry, dẫn đến code backend cực kỳ phức tạp và dễ sinh ra lỗi "trừ tiền ở ví rồi nhưng chưa ghi nhận học phí".

3. Một phản biện ngược lại

Một DBA (Database Administrator) có thể phản biện: *"Dùng Pessimistic Locking trong hệ thống trường đại học vào ngày chót đóng học phí sẽ tạo ra hàng ngàn Lock trên Database, dẫn đến hiện tượng Deadlock (khóa chéo) hoặc làm hệ thống chậm như rùa. Chúng ta nên dùng Optimistic Locking (chỉ dùng cột version hoặc updated_at) để tối đa hóa hiệu năng (throughput) của hệ thống. Nếu có xung đột, cứ báo lỗi cho sinh viên tự thao tác lại."*

Phản biện lại: Trong lĩnh vực tài chính, **tính chính xác quan trọng hơn hiệu năng**. Trải nghiệm người dùng sẽ tồi tệ hơn gấp 100 lần nếu hệ thống thông báo "Có lỗi xảy ra, vui lòng thử lại" nhưng tài khoản ngân hàng của họ thì đã bị trừ tiền, khiến họ không dám bấm thử lại nữa. Deadlock của Pessimistic Locking có thể được xử lý bằng cách giới hạn thời gian giữ khóa (Lock timeout) và tối ưu hóa câu lệnh query cho ngắn gọn nhất.

4. Một tình huống cụ thể

Tình huống "Cú click 30 triệu đồng":

Sinh viên B đang dùng 3G trên xe buýt để thanh toán học phí 10 triệu đồng. Mạng lag, ứng dụng báo "Đang xử lý...", sinh viên B sốt ruột **bấm thêm 2 lần nữa** vào nút "Thanh toán".

- **Nếu không có Idempotency:** Hệ thống nhận 3 request riêng biệt, thực thi 3 Database Transaction thành công. Sinh viên B bị trừ 30 triệu đồng.
- **Có Idempotency nhưng sai Locking:** Áp dụng Idempotency để chặn 3 request kia thành 1. Tuy nhiên, đúng giây phút đó, hệ thống học bổng cũng tự động chuyển 5 triệu vào tài khoản sinh viên B. Do thiếu Pessimistic Locking, luồng thanh toán và luồng học bổng cùng đọc số dư cũ, ghi đè lên nhau, làm thất thoát 5 triệu đồng của trường.

- **Giải pháp chuẩn:** Khi luồng thanh toán (có gắn Idempotency) bắt đầu chạy, nó dùng lệnh SQL SELECT ... FOR UPDATE (Pessimistic Locking) để khóa dòng dữ liệu của sinh viên B lại. Luồng cộng học bổng phải đứng chờ vài mili-giây. Sau khi thanh toán xong 10 triệu, mở khóa, luồng học bổng mới được phép cộng thêm 5 triệu trên số dư mới. Mọi thứ chính xác và an toàn tuyệt đối.